

Gerenciamento da CPU

Objetivos de Aprendizagem

- Como o SO fornece a ilusão de que vários programas estão rodando simultaneamente, mesmo com um número limitado de CPUs.
- Como os usuários podem usar as APIs do SO para iniciar e parar programas.
- Como o SO escolhe quais programas serão executados e quando alterná-los.

Abstração de Processo

- O conceito de **processo** é fundamental para virtualizar a CPU. Em uma determinada CPU, o SO pode iniciar um processo, pará-lo e executar outro.
- Com mecanismos adequados (como a **troca de contexto**) e políticas (como **algoritmos de agendamento**), o compartilhamento de tempo e a multiprogramação se tornam possíveis.

Definição de Processo

- Um **processo** é um programa em execução.

Definição Informal: Um programa pode criar vários processos.

- Um **programa** é uma entidade passiva, como um arquivo executável no disco com várias instruções.

Recursos do Sistema Utilizados por um Processo

- **CPU**: Para executar instruções de um programa.
- **Memória**: Para manter as instruções, variáveis e estruturas de dados de um programa.
- **Disco**: Para armazenar as informações do programa de forma persistente.

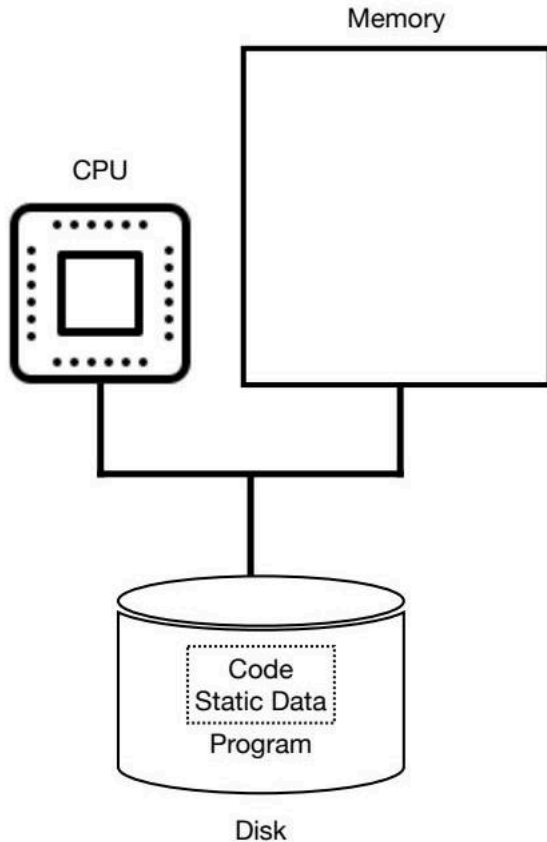
Representação da Atividade Atual de um Processo

- O valor do **Program Counter (PC)** e outros **registradores da CPU**: O PC é um registrador especial que identifica a próxima instrução a ser executada.
- **Espaço de endereço**: O layout da memória de um processo, onde residem as instruções ("código") e os dados (variáveis, estruturas, etc.) de um processo.
- **Lista de arquivos abertos**: Processos podem realizar operações de E/S.

Criação de Processos

Transformando Programas em Processos

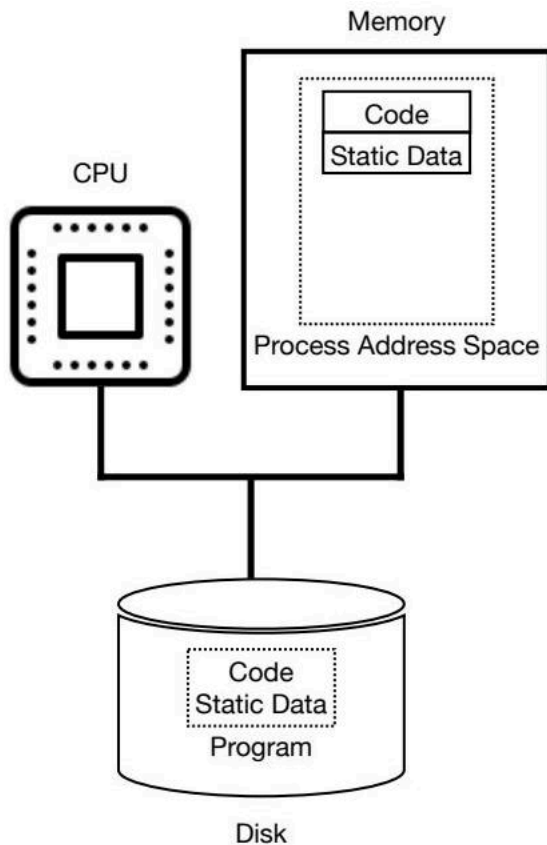
A imagem abaixo ilustra os componentes básicos de um sistema de computador, mostrando como a CPU, a memória e o disco interagem.



1. Os programas (código e dados estáticos) residem inicialmente no disco como arquivos executáveis.
2. Os arquivos de código-fonte dos programas são compilados em arquivos objeto (com a ajuda de um compilador).
3. Os arquivos objeto e as bibliotecas necessárias (por exemplo, a biblioteca C padrão) são combinados em um arquivo binário executável (com o linker).
 - O segmento de **código** também é conhecido como **texto**.
 - **Dados estáticos** (também conhecidos como segmento de dados) incluem, por exemplo, variáveis globais inicializadas.

Carregamento na Memória

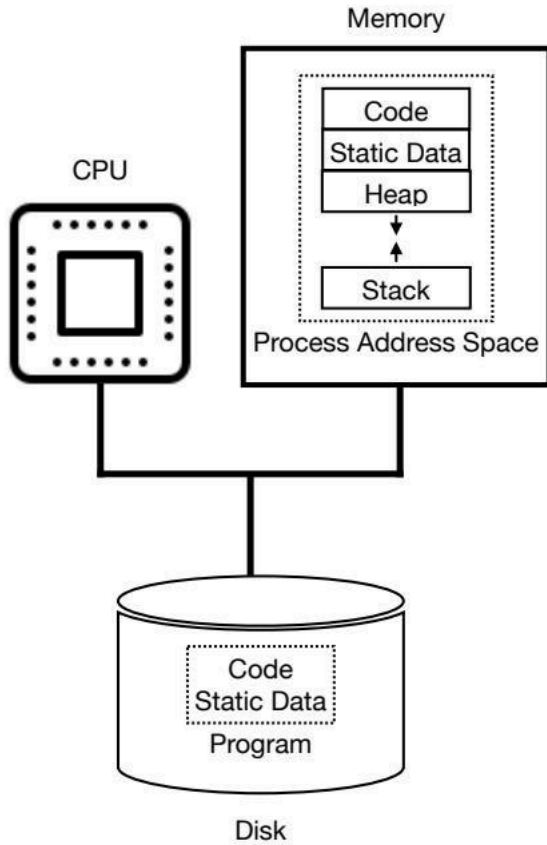
A imagem abaixo ilustra o fluxo de dados entre a CPU, a memória e o disco, mostrando como os programas são carregados na memória.



- Bytes do arquivo binário (armazenados no disco) são carregados na memória (com o loader).
 - O carregamento pode ser feito **eagerly** (todos os bytes são carregados antes de executar o programa) ou **lazily** (pedaços do programa são carregados conforme são necessários e enquanto o programa está em execução). Sistemas operacionais modernos usam carregamento lazy.

Alocação de Memória

A imagem a seguir mostra como o código, os dados estáticos, o heap e a pilha são organizados no espaço de endereço do processo.

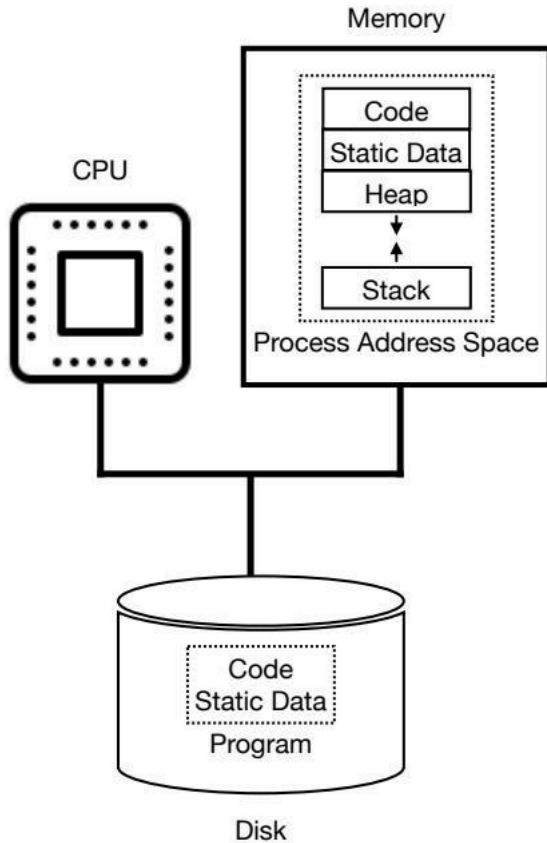


Uma vez que o código e os dados estáticos são carregados na memória, o SO:

- Aloca memória para a **pilha** (por exemplo, argumentos, variáveis locais, parâmetros de função, endereços de retorno).
- Aloca memória para o **heap** (memória alocada dinamicamente).
- Realiza outras tarefas relacionadas a E/S (por exemplo, inicializa descritores de arquivo padrão para stdin, stdout e stderr).

Execução do Programa

Esta imagem ilustra a organização da memória e a interação entre CPU, memória e disco.



- O SO transfere o controle da CPU para o processo recém-criado para começar a executar no ponto de entrada do programa (rotina `main()`).

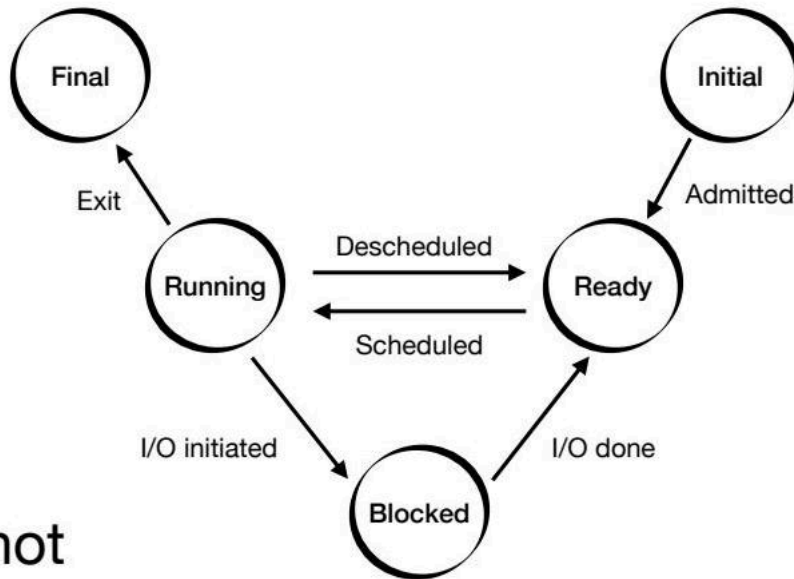
Estados do Ciclo de Vida de um Processo

Um processo pode estar nos seguintes estados:

- **Executando (Running)**: O processo está sendo executado no processador.
- **Pronto (Ready)**: O processo está aguardando tempo de processador.
- **Bloqueado (Blocked)**: O processo está aguardando que outro evento ocorra (por exemplo, uma solicitação de E/S de disco).

Estados Adicionais

O diagrama a seguir ilustra os diferentes estados de um processo e as transições entre eles.

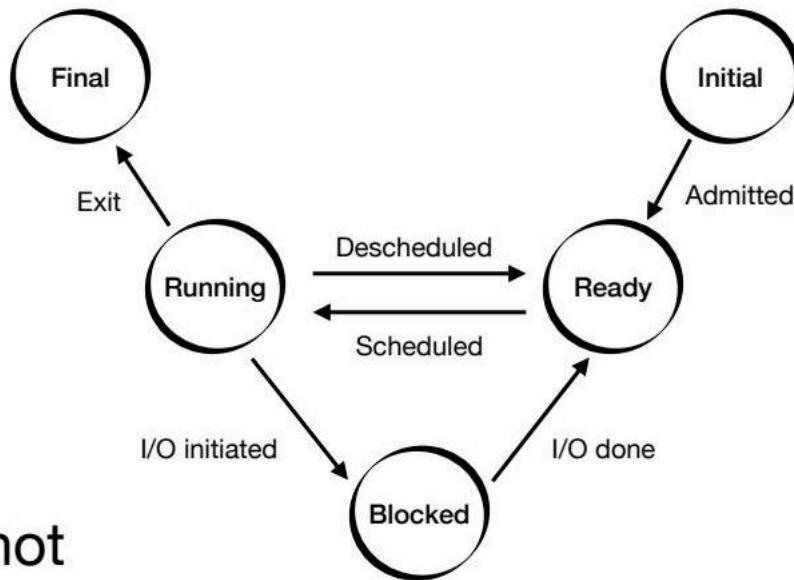


- **Initial (Initial):** O processo está sendo criado.
- **Final (Final):** O processo terminou a execução, mas ainda não foi limpo (estado zumbi no Unix).

Alguns sistemas operacionais podem delinear seus estados ainda mais.

Transições de Estado

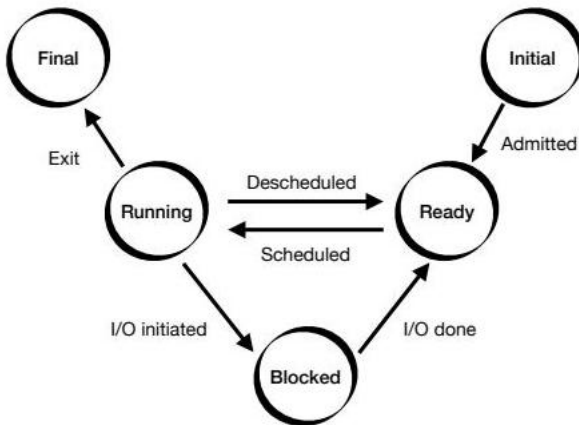
Este diagrama fornece uma representação visual clara do ciclo de vida do processo.



- Durante a execução, um processo pode transitar várias vezes pelos estados Running, Ready e Blocked.

Exemplo de Transições de Estado

O diagrama a seguir exemplifica as transições entre os estados de um processo.



- Processo 0 bloqueia para E/S e o Processo 1 é agendado (tempo 4).
- Processo 0 está pronto, mas o SO decide manter o Processo 1 em execução (tempo 7).

Decisões do Agendador do SO

- Por que é uma boa ideia agendar outro processo quando o Processo 0 bloqueia para E/S (tempo 4)?
- Deve-se agendar imediatamente o Processo 0 quando a E/S é concluída (tempo 7)?

Essas decisões são tomadas pelo [agendador do SO](#).

Estruturas de Dados do Sistema Operacional

Lista de Processos e PCB

- O SO é um programa, portanto, usa estruturas de dados!
- Uma [lista de processos](#) é usada para rastrear o estado dos processos (ou seja, Running, Ready, Blocked).
- Cada entrada desta lista é chamada de [Process Control Block \(PCB\)](#).

Componentes do PCB

- [Estado do Processo](#): Running, Ready, Blocked, etc.
- [Process ID \(PID\)](#): Identificador único para o processo.
- [Program Counter](#): Endereço de memória da próxima instrução a ser executada.
- [Registradores](#): Registradores da CPU que devem ser salvos na memória para que o processo possa ser retomado posteriormente (por exemplo, ponteiros de pilha).
- [Limites de Memória](#): Os limites de memória (espaço de endereço) que o processo pode acessar.
- [Lista de Arquivos Abertos](#): Arquivos abertos pelo processo.

API de Processos

Funções da API

A API deve permitir:

- Criar novos processos.
- Destruir processos.
- Esperar por um processo até que ele termine sua execução.
- Verificar o status de um processo (por exemplo, seu estado atual, tempo de execução).
- Outros controles, como suspender e retomar um processo sob demanda.

API de Processos UNIX

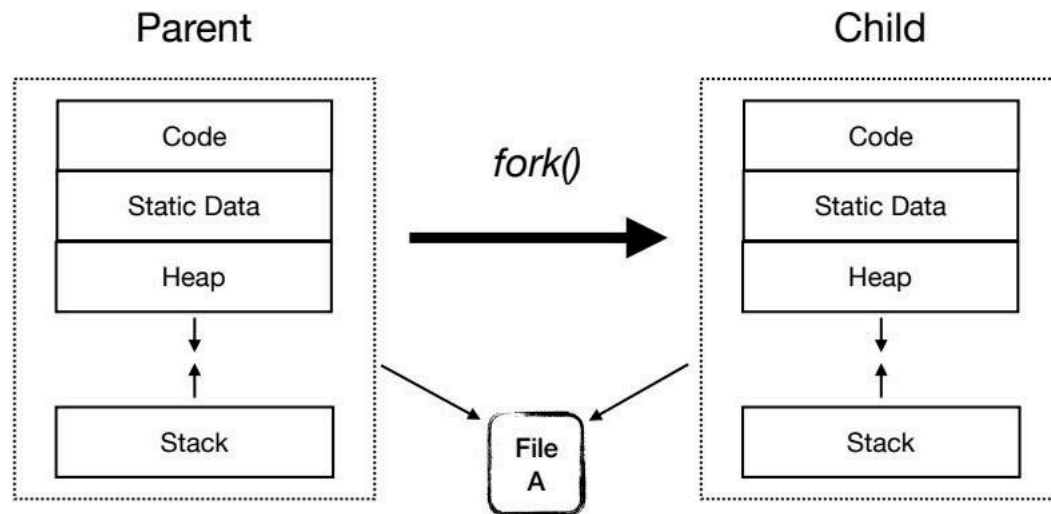
Fork

- A chamada de sistema `fork()` é usada para criar um novo processo.

```
pid_t fork(void)
```

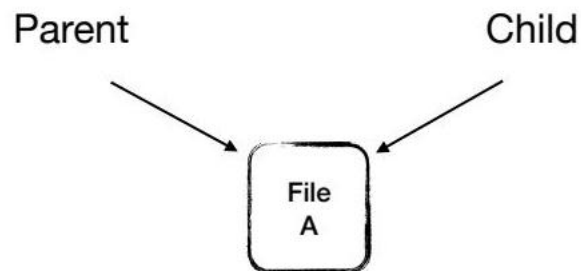
- Quando `fork` é chamado pelo processo pai, o processo filho (uma cópia quase idêntica do processo que chama) é criado.
- O processo filho tem sua própria cópia do espaço de endereço, registradores, PC e lista de arquivos abertos.
 - Copiar != Compartilhar**

A imagem a seguir ilustra o processo de criação de um processo filho a partir de um processo pai usando `fork()`.



Valor de Retorno de Fork

O diagrama abaixo ilustra a relação pai-filho e o compartilhamento de arquivos.



- O valor retornado por `fork()` para cada processo é diferente:
 - O pai recebe o PID do filho recém-criado.
 - O filho recebe um código de retorno de zero.
- Dois processos estão agora rodando simultaneamente, acessando um espaço de endereço independente (memória).
 - O que acontece se os dois processos escrevem no arquivo A?
- Dois processos estão agora executando as instruções do mesmo programa.
 - Como especificar código a ser executado por cada processo?

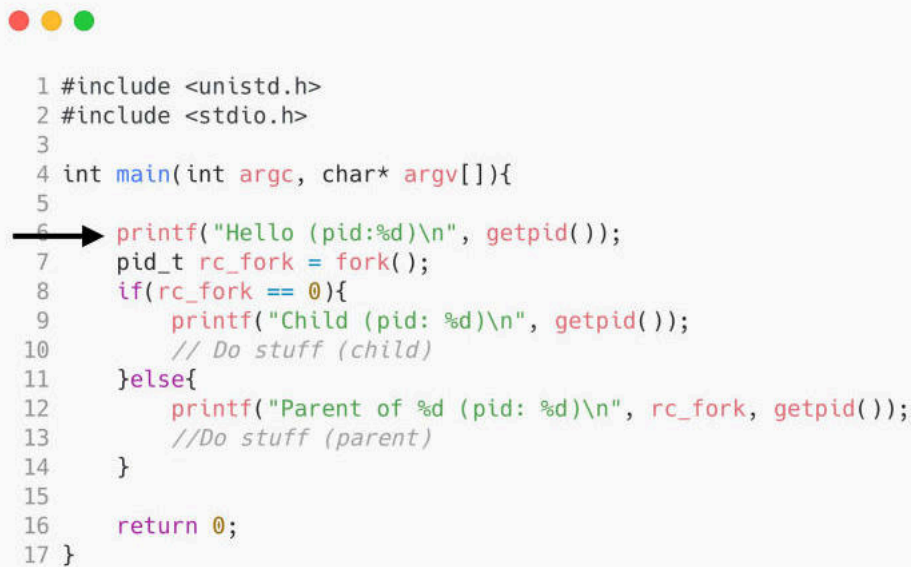
Exemplo de Fork

- Compile e execute este código!


```
prompt> ./p1
```

Exemplo de Saída

Este diagrama mostra um exemplo de saída de um programa que usa `fork()`.



```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main(int argc, char* argv[]){
5
6   printf("Hello (pid:%d)\n", getpid());
7   pid_t rc_fork = fork();
8   if(rc_fork == 0){
9       printf("Child (pid: %d)\n", getpid());
10      // Do stuff (child)
11   }else{
12       printf("Parent of %d (pid: %d)\n", rc_fork, getpid());
13       //Do stuff (parent)
14   }
15
16   return 0;
17 }
```

```
prompt> ./p1
Hello (pid:44562)
Prompt
```

- Quando o programa começa, o processo imprime a mensagem "Hello" e seu PID (44562).

Criação de Processo Filho

Este diagrama demonstra a criação de um processo filho usando a chamada de sistema `fork()`.

```

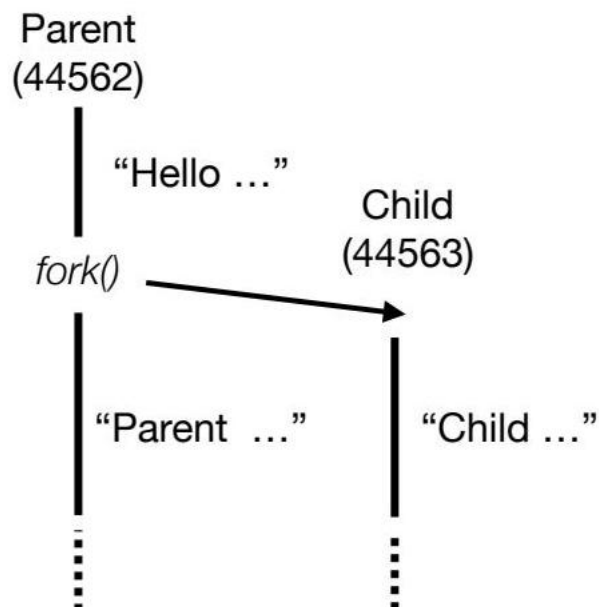
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main(int argc, char* argv[]){
5
6   printf("Hello (pid:%d)\n", getpid());
7   pid_t rc_fork = fork();
8   if(rc_fork == 0){
9       printf("Child (pid: %d)\n", getpid());
10      // Do stuff (child)
11  }else{
12      printf("Parent of %d (pid: %d)\n", rc_fork, getpid());
13      //Do stuff (parent)
14  }
15
16  return 0;
17 }

```

- Quando `fork()` é chamado, um novo processo (filho) é criado, que é uma cópia quase idêntica do processo que chama (pai).
- O filho não começa a rodar em `main()`; ele começa como se tivesse chamado `fork()` ele mesmo ("Hello" é impresso apenas uma vez).
- O filho não é uma cópia idêntica, pois recebe um código de retorno diferente de `fork()`. Isso é útil para especificar código diferente (realizar ações diferentes) para cada processo.

Concorrência com Fork

O diagrama a seguir ilustra a concorrência entre processos pai e filho criados com `fork()`.



```
prompt> ./p1
Hello (pid:44562)
Parent of 44563 (pid: 44562)
Child (pid: 44563)
prompt>
```

- Se o programa anterior for executado várias vezes, as mensagens "Parent..." e "Child..." podem ser trocadas. Por quê?
- Mesmo assumindo uma única CPU, a saída não é determinística. Por quê?

Não Determinismo e Concorrência

- O não determinismo leva a problemas interessantes de concorrência.
- Embora a memória de cada processo seja isolada (privada), os processos podem compartilhar acesso a outros recursos (por exemplo, arquivos).
- A memória não é isolada para programas multi-threaded. Threads de um processo têm seu próprio contador de programa, mas compartilham o espaço de endereço do processo.

Wait na API de Processos UNIX

Esperando pelo Término do Filho

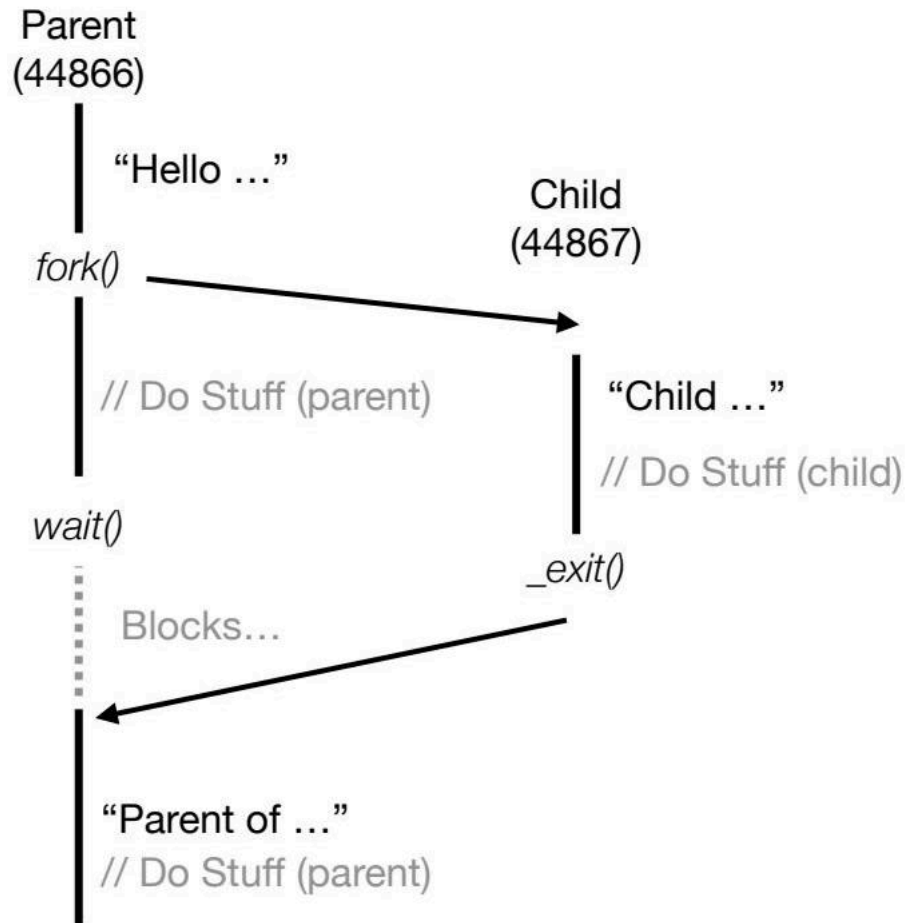
- E se o pai quiser esperar que seu filho termine de executar?
- A chamada de sistema `wait()` bloqueia a execução do processo que chama até que um dos processos filhos termine.

```
pid_t wait(int *status);
```

- Algumas variantes de `wait()` podem não bloquear até que o processo filho saia.

Exemplo de Saída com Wait

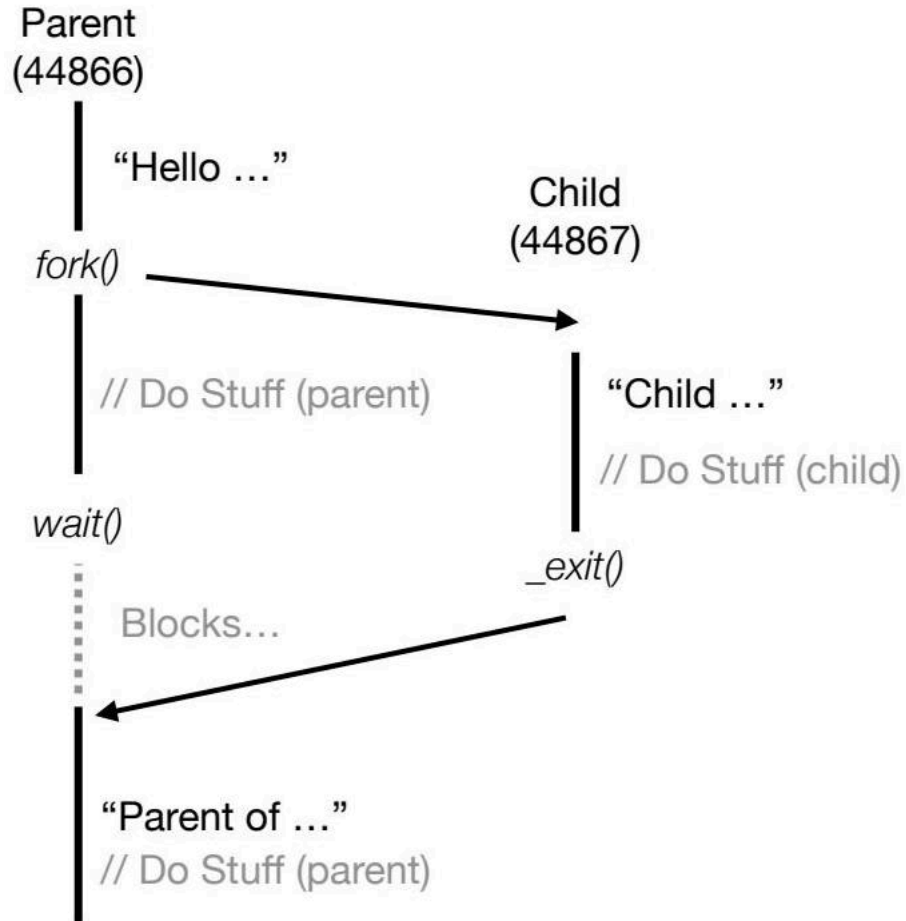
Este diagrama ilustra o uso da função `wait()` para sincronizar processos pai e filho.



```
prompt> ./p1
Hello (pid:44866)
Child (pid: 44867)
Parent of 44867 (rc_wait: 44867) (pid: 44866)
prompt>
```

Fork e Wait Concorrente

O seguinte diagrama ilustra a sincronização entre os processos pai e filho usando `fork()` e `wait()`.



- Se o programa anterior for executado várias vezes, as mensagens "Child..." e "Parent..." são sempre impressas na mesma ordem. Por quê?

```

prompt> ./p1
Hello (pid:44866)
Child (pid: 44867)
Parent of 44867 (rc_wait: 44867) (pid: 44866)
prompt>

```

Outros Detalhes e Laboratórios Práticos

Tratamento de Erros e Funções Wait

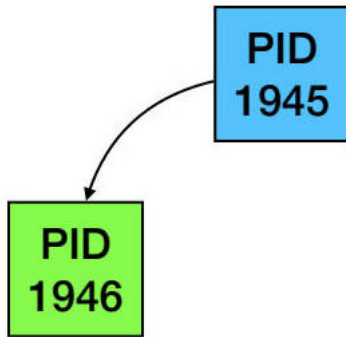
- Não se esqueça de tratar os erros (quando as chamadas de sistema retornam -1)!
- Consulte a página [man](#) de `wait()`:
 - Explore o argumento `status` e as macros `WIFEXITED` e `WEXITSTATUS`.
 - Explore a chamada `waitpid()`.

Processos Zumbis e Órfãos

- O que acontece se o filho sai antes que o pai chame `wait()`?
 - O filho entra em um estado zumbi (aguardando que o pai colete seu status).
- O que acontece se o pai sai enquanto o filho ainda está executando?
 - O filho se torna órfão, sendo adotado por outro processo (por exemplo, o processo `init`).

Número de Processos Criados

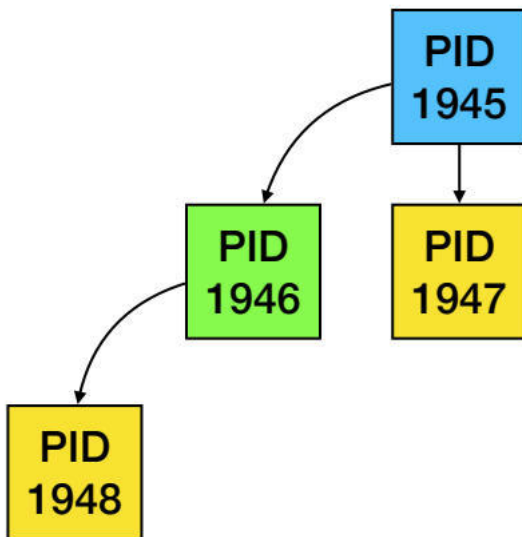
O diagrama abaixo representa a criação de um processo filho.



- Quantos processos o código abaixo cria?
- Você pode desenhar a árvore de processos?

Árvore de Processos

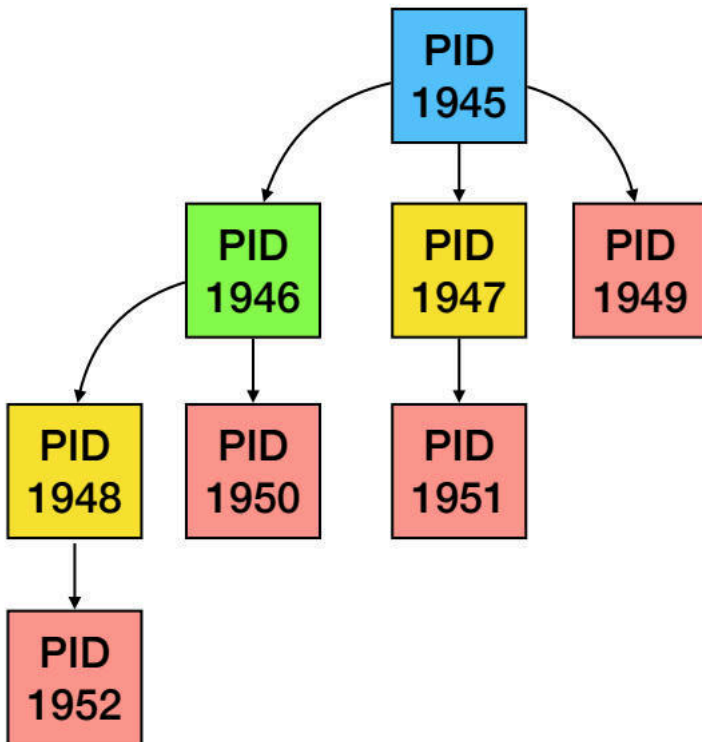
Este diagrama ilustra a estrutura de uma árvore de processos.



- Quantos processos o código abaixo cria?
- Você pode desenhar a árvore de processos?

Árvore de Processos Completa

O diagrama a seguir mostra uma árvore de processos mais complexa.



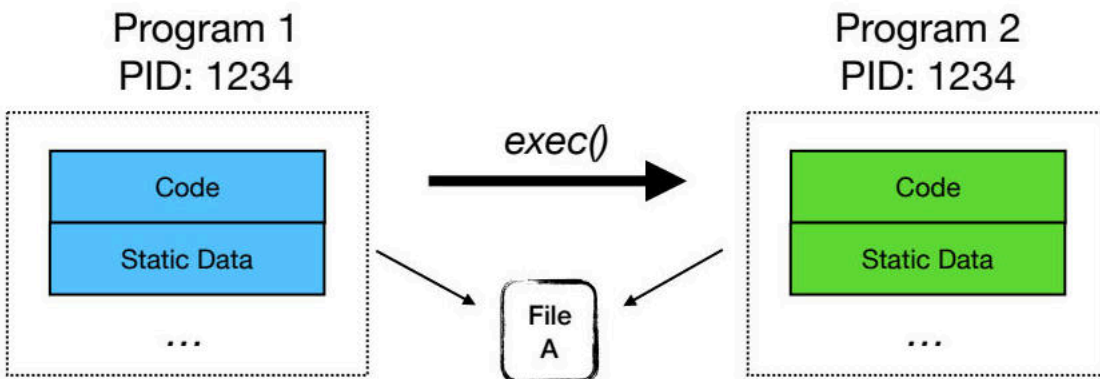
- Quantos processos o código abaixo cria?
- Você pode desenhar a árvore de processos?

API de Processos UNIX - Exec

Substituindo o Programa do Filho

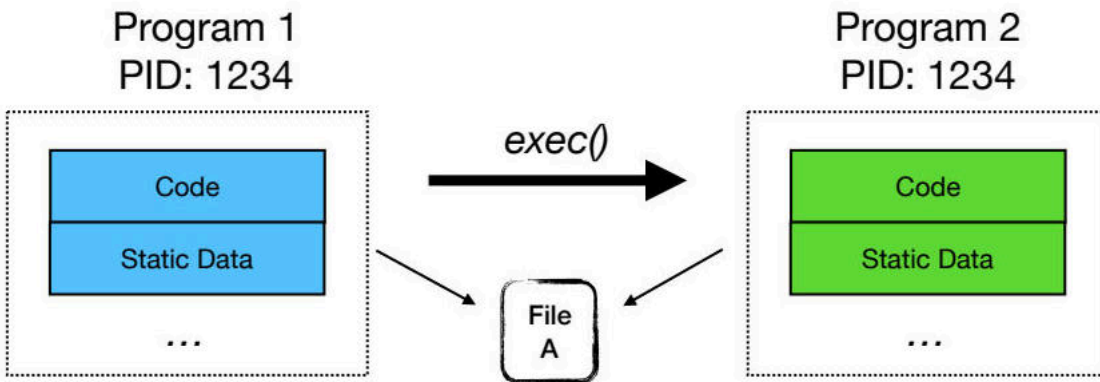
- E se o filho quiser executar um programa diferente?
- A chamada de sistema `exec()` substitui o código e os dados estáticos do processo atual pelos de um novo programa.
 - Heap e pilha são reinicializados.
 - O novo programa começa no ponto de entrada `main()`.
 - O PID é mantido (mesmo processo).
 - Os recursos inicializados (por exemplo, arquivos abertos) ainda são utilizáveis pelo processo.

O diagrama a seguir ilustra a substituição de um processo por outro usando `exec()`.



Variantes de Exec

Existem várias variantes de `exec()` (por exemplo, `execv()`, `execp()`, `execvp()`, ...). Este diagrama representa o processo de substituir o programa atual por um novo programa usando a função `exec()`.



Exemplo de Exec

```
int execvp(const char *file, char *const argv[]);
```

- `file`: caminho/nome do programa.
- `argv`: lista de argumentos do programa (terminada com NULL).
- O programa `wc` é executado no processo filho.

O diagrama abaixo mostra a saída do comando `wc`.

Output of `wc` →

```

prompt> ./p1
Child (pid: 77262)
23  77  648 exec.c
Parent of 77262 (pid: 77261)
prompt>
  
```

Execução do Comando wc

```
Try running the following command in your shell: wc exec.c
```

- `exec()` só retorna ao programa original se não for bem-sucedido.

Este diagrama ilustra a saída do comando Unix `wc`.

Output of `wc` →

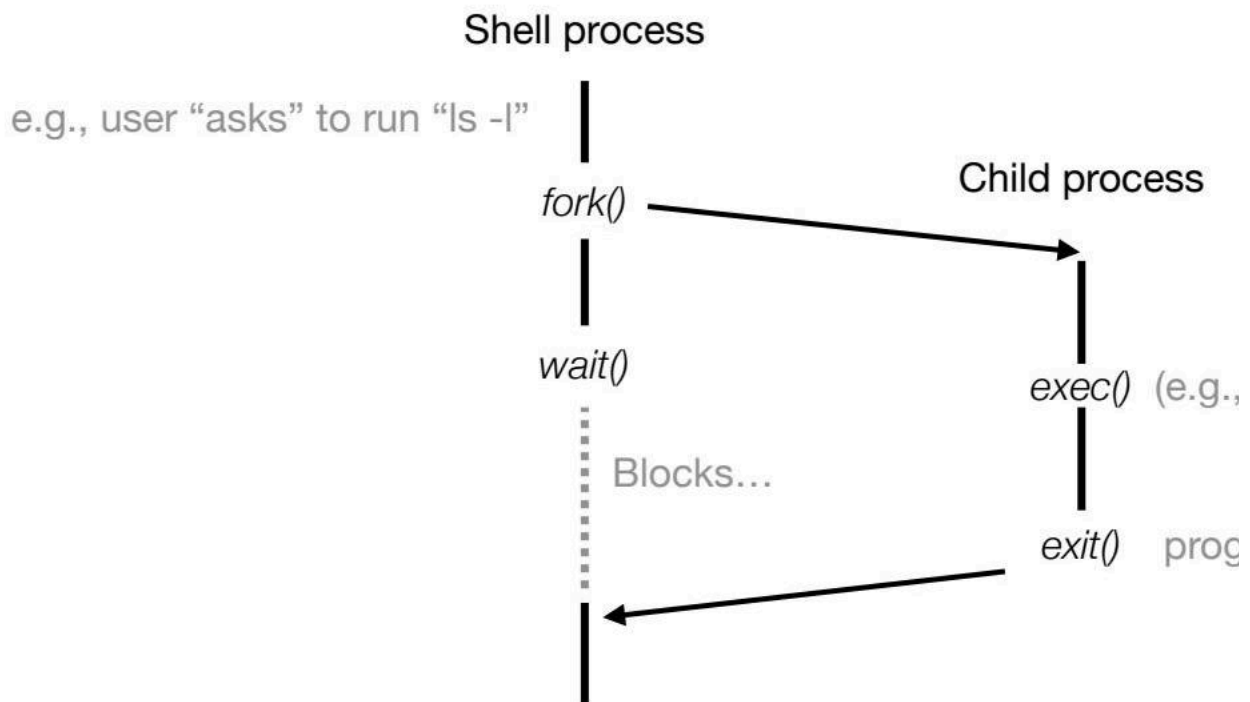
```

prompt> ./p1
Child (pid: 77262)
23   77   648 exec.c
Parent of 77262 (pid: 77261)
prompt>

```

Uma Combinação Poderosa: Fork, Exec, Wait

O diagrama abaixo mostra um exemplo de como um shell executa comandos usando `fork()`, `exec()` e `wait()`.



- Como o shell (bash) executa seus programas?

Árvore de Processos em Unix e Linux

Este diagrama ilustra uma árvore de processos em um sistema Unix ou Linux, mostrando a relação entre os processos `init`, `bash` e `wc`.

